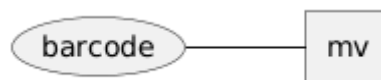


Modelo Entidad Relación (MER)

Permite conceptualizar bases de datos describiendo entidades, atributos y las relaciones entre estos. Pueden ser plasmados a través del lenguaje UML (Unified Modeling Language) y por tanto, a través de PlantUML.

Haciendo un diagrama de entidad-relación en PlantUML

- `@startchen/@endchen` Abren y cierran, respectivamente, un diagrama de este tipo.
- Por defecto, PlantUML extiende de manera vertical los diagramas. Para designar una extensión horizontal, después de abrir el diagrama, escribir `left to right direction`.
- La manera de representar entidades y sus atributos es similar al formato clave-valor de JSON (uso de brackets, `clave:valor`)
- `columna` = campo = atributo; `fila` = registro = instancia; `entidad` = tabla
- Puede asignarse un alias a entidades, atributos y relaciones



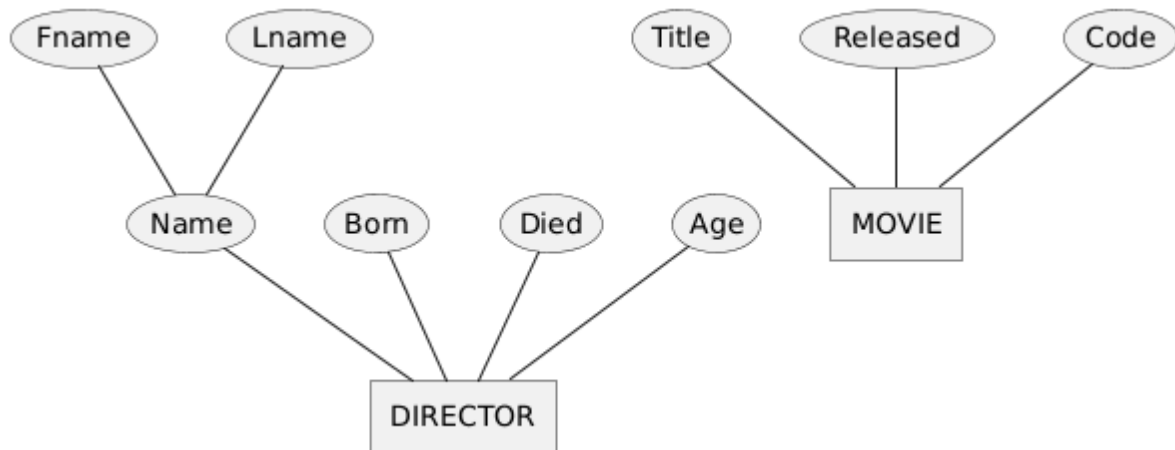
Pasos para hacer el MER:

- 1) Lee el problema/necesidad del sistema de información
- 2) Enlista los sustantivos, adjetivos, verbos, propiedades y posibles relaciones.
- 3) Determina qué atributos pertenecen a las entidades y relaciones
- 4) Determina la cardinalidad de las relaciones
- 5) Realiza la asignación de llaves dada la cardinalidad
- 6) Busca restricciones que no se puedan reflejar en el MER

Retos al hacer un MER:

- 1) Distinguir entidades de atributos
- 2) Asignar atributos a entidades y relaciones
- 3) Identificar la cardinalidad de relaciones
- 4) Identificar las relaciones entre entidades
- 5) Entender la necesidad del cliente
- 6) Abstractar la realidad

Entidad - Referible mediante sustantivos



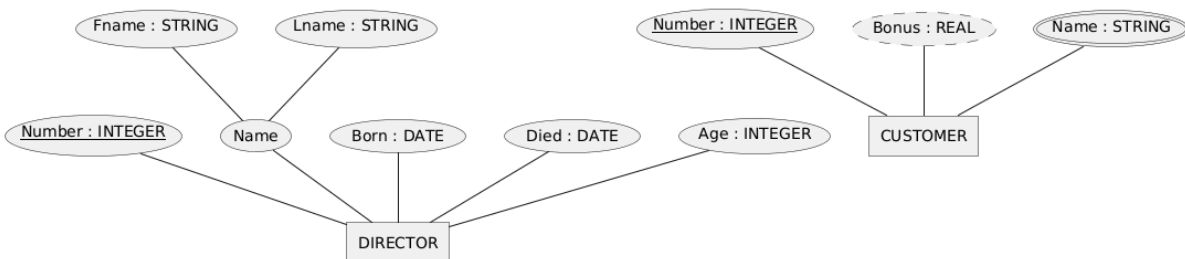
(*entity \$NAME\$ {...}, rectángulo*)

Útiles para representar seres, objetos o criaturas y contener sus atributos esenciales, de interés. Son

Tipos:

- **Débil** (*entity \$NAME\$ <<weak>> {...}, rectángulo doble*) si su existencia depende de otra entidad.
- **Recursiva** si tiene relación consigo misma.

Atributo - Referible mediante adjetivos



(*\$ATTRIBUTENAME\$: \$ATTRIBUTEVALUE\$, óvalo*)

Representa una propiedad de entidad.

Su valor tiene las siguientes características:

- Si se origina de otro atributo, **atributo derivado** (<<derived>>, óvalo punteado)
- Puede consistir en múltiples valores (<<multi>>, óvalo doble)
- Una llave también es un atributo (<<key>>, texto subrayado)

Criterios para poner atributos:

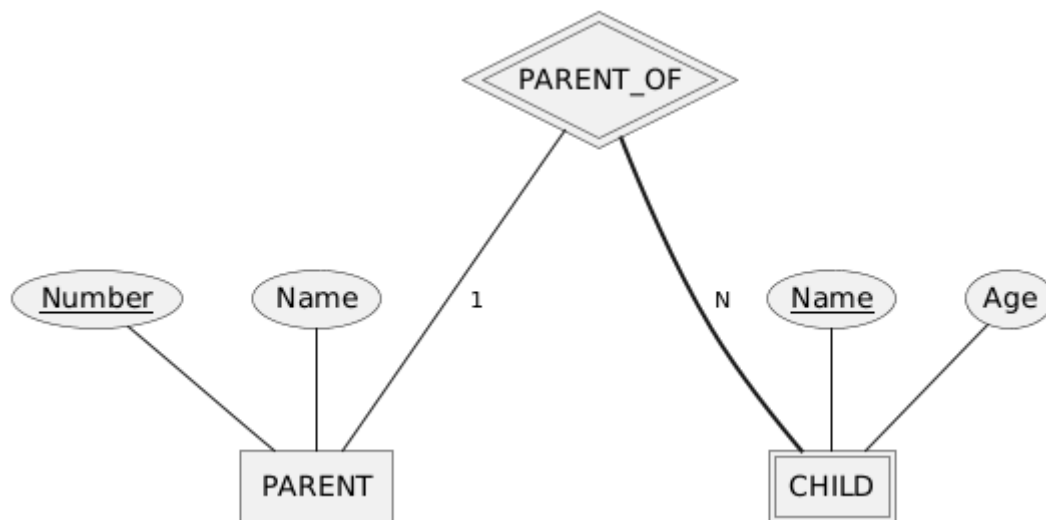
- Piensa en la pertenencia del atributo
- ¿El atributo es del objeto o de la acción que realiza el objeto?

Llaves

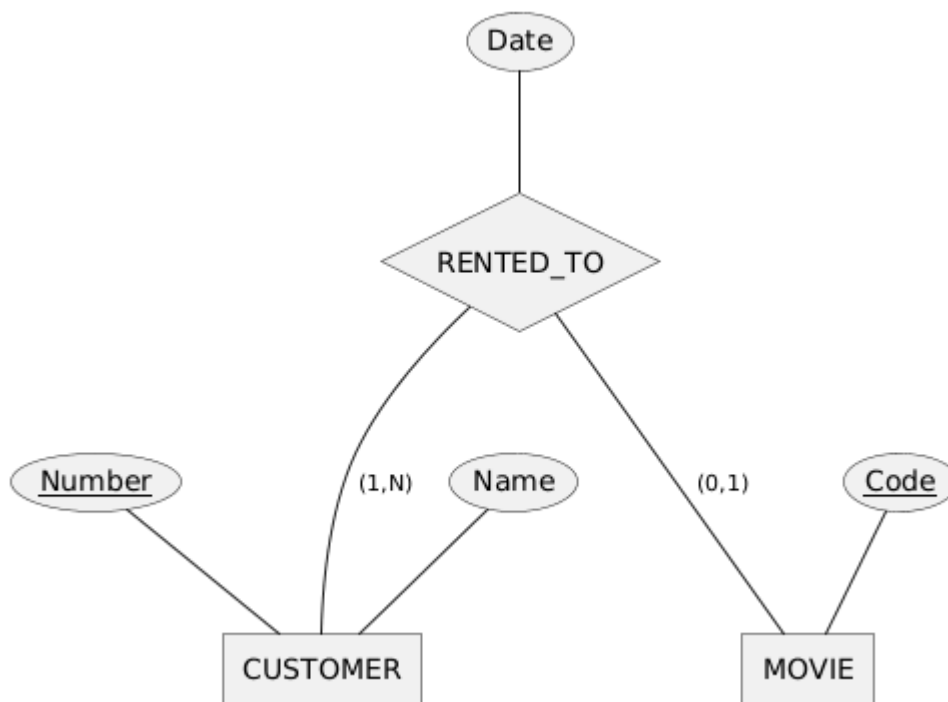
Atributo que **identifica** directamente (**llave primaria, PK**) a una entidad de manera (única, no nula y no ambigua). De no haberlo, se le crea (**llave primaria artificial, APK**).

Dicho identificador es usado al referenciar a su entidad de manera externa (**llave foránea, FK**)

* La entidad débil no contiene llave primaria. Le identifica una concatenación de algún atributo propio y la PK de su entidad fuerte.



Relación - Referible mediante verbos



(relationship \$NOMBRE_RELACION\$ {...}, rombo)

Útiles para representar la interacción, dependencia, o asociación entre **N** entidades, donde **N** es el grado de la relación.

Participación

Enlaza una relación con entidades, **interconectándolas**. Puede ser total/obligatoria (**= \$CARDINALIDAD\$ =**, línea gruesa) o parcial/opcional (**- \$CARDINALIDAD\$ -**, línea delgada)

* Una entidad débil tiene participación total con la relación (**relación identificadora**) que le conecta con su entidad fuerte.

Cardinalidad

La cardinalidad (**(N, N)**, rango) especifica entre cuántas instancias se efectúa una participación. Considera la direccionalidad (sobre el tiempo) de la participación entre las instancias de las entidades involucradas.

Tipos:

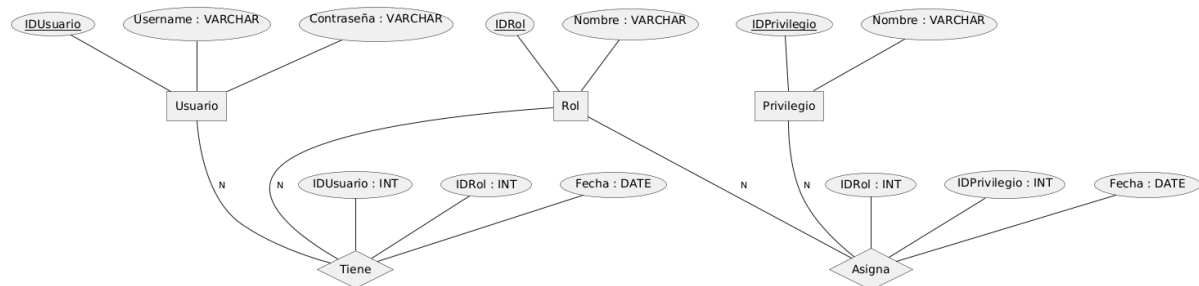
- Uno a uno (1:1): Las entidades heredan sus llaves primarias entre sí
- Uno a muchos (1:N): La entidad (N) hereda la llave primaria de la entidad (1)

- Muchos a muchos (N:N): Se añaden atributos en la relación (entre ellos la llave primaria de ambas entidades y quizá una llave primaria artificial para la relación que sirva para enlistarla y dejar las otras llaves como foráneas)
- Definida (por ejemplo, 5:3)

MEER (MER Extendido)

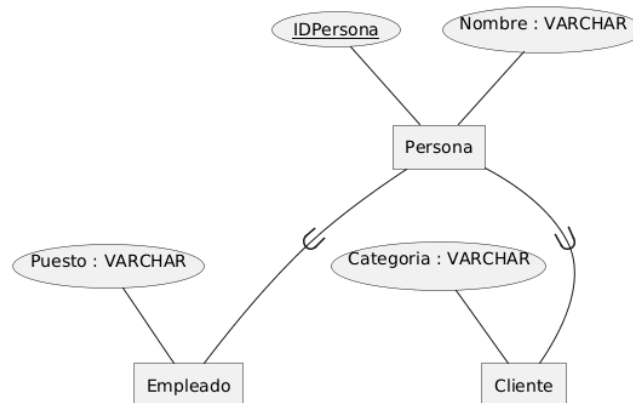
Notación extendida del MER, con soporte para subclasses y tipos de unión.

RBAC (Control de Acceso Basado en Roles)



Modela los roles y permisos de un usuario a los recursos de un sistema. Permite a una entidad jugar más de un rol en cualquier relación. Se representa a los roles como entidades interconectadas con las entidades a contenerlos a través de la relación 'Tiene'.

Relación ISA



Permite modelar relaciones padre-hijo (especializaciones) sobre entidades.

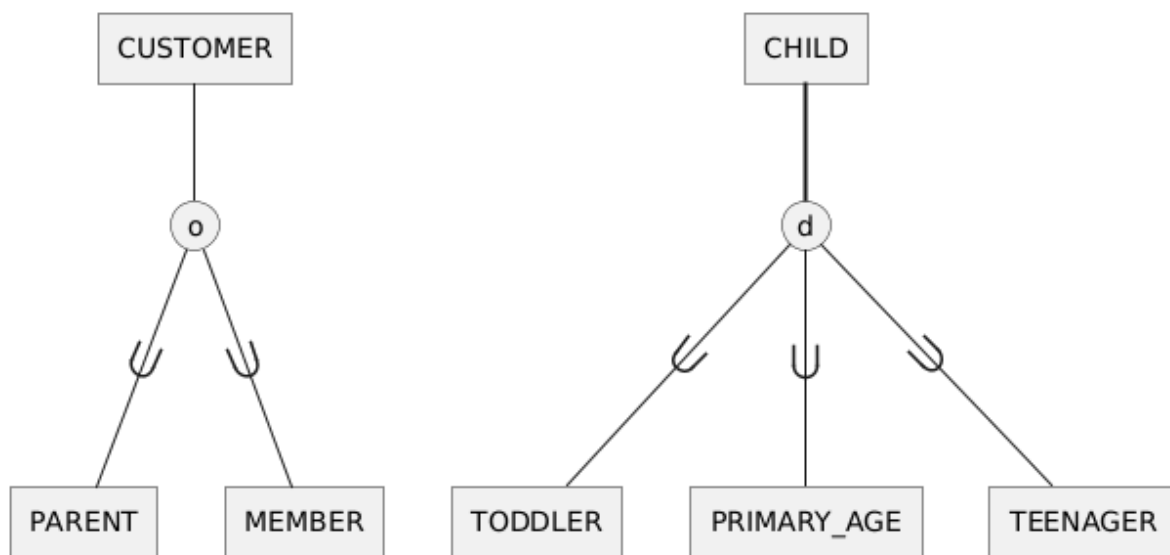
- Se representan por un triángulo
- Tiene una cardinalidad 1:1
- Las entidades hijo heredan la llave primaria de la entidad padre

Roles	ISA
Relaciones entre entidades iguales	Jerarquía entre entidades

Roles	ISA
Las entidades en cada rol tienen los mismos atributos (entidades iguales)	Las entidades heredan atributos, pero tienen sus propios atributos
Multiplicidad de entidades	Especialización de entidades

Subclases y categorías

Las entidades pueden tener subclases y superclases



Modelo Relacional (MR)

Permite representar bases de datos a través de tablas, atributos y las relaciones entre estas. Pueden ser plasmados a través del lenguaje UML (Unified Modeling Language) y por tanto, a través de PlantUML.

1. Relación entre MER y MR

Para adaptar lo representado por un MER y expresarlo en términos de tablas.

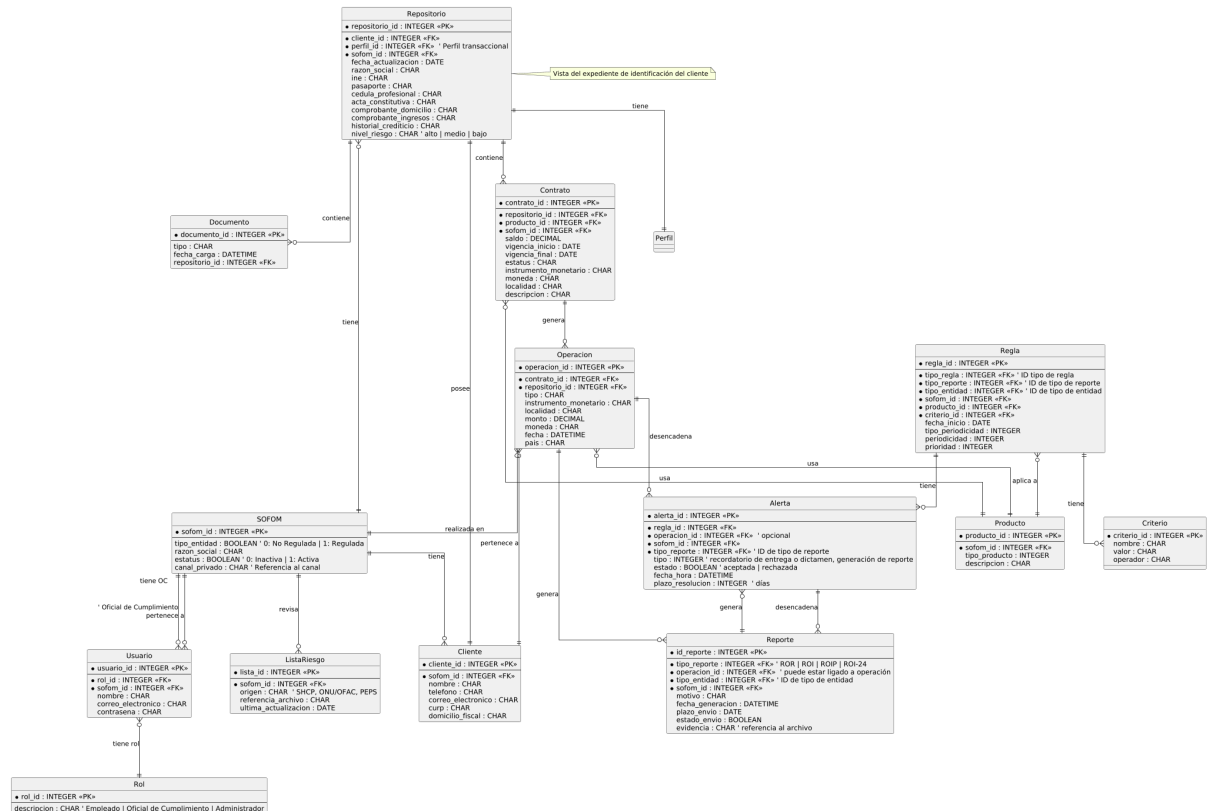
2. Reglas generales de traducción

Elemento del MER	En el MR se convierte en	Descripción
Entidad fuerte (simple)	Una tabla	Cada entidad = una tabla con sus atributos

Elemento del MER	En el MR se convierte en	Descripción
Entidad débil	Una tabla que hereda la PK de su entidad fuerte	Su PK = PK de la entidad fuerte + su propio identificador parcial
Atributos simples	Columnas	Se convierten directamente
Atributos compuestos	Una nueva tabla	Incluye los sub-atributos y hereda la PK de la entidad original
Atributos multivalor	Una nueva tabla	Contiene la PK de la entidad y el valor múltiple
Atributos derivados	No se representan	Se calculan con consultas
Relación 1:1	Ambas entidades intercambian PK como FK	A veces se fusionan si tiene atributos
Relación 1:N	La entidad del lado "N" hereda la PK de la del lado "1" como FK	Se añade la FK en la tabla (N)
Relación N:N	Se crea una tabla intermedia (relación)	Tiene como PK la unión de las PK de las entidades participantes
Relación recursiva	Una FK que apunta a la misma tabla	Ej: <code>Empleado(SupervisorID)</code>
Relaciones con roles	Igual que las recursivas, pero se renombra cada FK según el rol	
Relación ISA (herencia)	Las tablas hijas heredan la PK de la tabla padre	Mantienen la misma llave
Participación total/parcial	Se refleja con restricciones de obligatoriedad (NOT NULL)	No cambia la estructura del MR

Un enfoque más práctico

Una notación para representar MER y MR en conjunto, a través de un solo diagrama, es utilizando la notación de PlantUML para diagramas de relación de entidades. Esta notación del MER, por su formato de tablas interconectadas por relaciones con cardinalidad, y el hecho de que es extensible para ahondar en atributos (y los detalles de estos), permite conseguirlo.



Con esta notación y formato, el MER y MR pueden ser representados como un solo diagrama. El diccionario de datos permite especificar longitudes y tipos de datos correspondientes para algún DBMS en particular.

Además, en lo particular considero que es un diagrama más limpio y aún así más completo.

Diagramas de secuencia

Diagrama de comportamiento del UML que muestra la interacción entre objetos a lo largo del tiempo, enfatizando el orden temporal de la comunicación para un escenario o caso de uso.

Ejes y lectura

- Horizontal: objetos/instancias (colocados en la parte superior).

- Vertical: tiempo (de arriba hacia abajo); el orden vertical indica la secuencia de ejecución.

Elementos principales

- **Objeto:** instancia de clase participante.
- **Línea de vida:** línea vertical discontinua que representa la existencia del objeto.
- **Foco de control:** rectángulo delgado sobre la línea de vida que indica ejecución activa.
- **Mensajes:** flechas entre líneas de vida que representan comunicación.

Tipos de mensajes

- **Síncrono:** llamada a método; emisor espera respuesta — flecha con cabeza llena.
- **Asíncrono:** no espera respuesta; ejecución continúa — flecha con cabeza abierta.
- **Retorno:** respuesta; línea discontinua (opcional si es evidente).
- **Creación/Destrucción:** <> y <> marcando inicio/fin de la línea de vida.

Variantes de diagrama

- **De instancia:** escenario concreto (una ejecución).
- **Genérico:** describe el comportamiento completo de un caso de uso (incluye condiciones, ciclos y alternativas).

Control de flujo: fragmentos combinados

- **alt:** alternativas (if–else).
- **opt:** condición opcional.
- **loop:** bucle mientras se cumpla la condición.
- **par:** ejecución paralela.
- **critical:** sección crítica (exclusión mutua).

Utilidad

- Facilita entender interacciones entre objetos.
- Valida la lógica y ayuda a pasar del diseño conceptual a la implementación.
- Complementa casos de uso y diagramas de clases.

Ejemplo del proyecto para SVA Law (en torno a la dinámica de parametrización de reglas)

